Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

ОТЧЕТ
по лабораторной работе №3
по дисциплине
«Администрирование и программирование распределенных приложений»
Вариант 2

Выполнила: ст. гр. 320604
К.А. Хаджинова

Проверила: Ю.О. Герман

Минск 2026

СОДЕРЖАНИЕ

1 ЦЕЛЬ РАБОТЫ 3
2 ХОД РАБОТЫ 4
2.1 Постановка задачи 4
2.2 Теоретические сведения 4
2.3 Подготовка среды и структура проекта 5
2.4 Страница *dictionary.html* 5
2.5 Сервлет *MyServlet* и база данных переводов 6
2.6 Работа словаря 6
2.7 Клиент в отдельном проекте 7
2.8 Индивидуальное задание, вариант 2 9
3 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ 10
ВЫВОДЫ 11
ПРИЛОЖЕНИЕ А (ОБЯЗАТЕЛЬНОЕ) ЛИСТИНГИ ПРОГРАММНОГО КОДА 12

1 ЦЕЛЬ РАБОТЫ

Изучить серверные классы *Java* (сервлеты): структуру веб-приложения, обработку запросов методами *doGet* и *doPost*, регистрацию сервлетов в файле *web.xml*, развертывание приложения на сервере *Apache Tomcat*. Научиться обращаться к сервлету из клиентской страницы и из отдельного клиентского приложения и выполнять индивидуальное задание варианта 2 – сервлет, возвращающий клиенту рисунок.

2 ХОД РАБОТЫ

Работа выполнена на языке *Java* 17 с использованием сервлетов (*Servlet API* 4.0), сборки *Maven* и сервера *Apache Tomcat* 9. Полный текст программы приведен в приложении А. Ниже последовательно описаны постановка задачи, теоретическая основа, реализация заданий и результаты проверки.

2.1 Постановка задачи

Для изучения серверных классов в рамках работы требуется выполнить следующие задачи:

1 создать веб-приложение со страницей *dictionary.html* и сервлетом перевода слов;

2 хранить переводы в базе данных и возвращать результат клиенту;

3 зарегистрировать сервлеты в *web.xml* и развернуть приложение на *Tomcat*;

4 создать клиента в отдельном проекте, обращающегося к сервлету;

5 выполнить индивидуальное задание варианта 2: сервлет, возвращающий клиенту рисунок.

2.2 Теоретические сведения

Сервлет – класс *Java*, который выполняется на стороне сервера и обрабатывает запросы клиентов. Клиентская часть обычно представляется сайтом (*html*-документом). Форма сайта вызывает сервлет по имени, указанному в атрибуте *action* тега *form*, а метод передачи данных задается атрибутом *method*: *Get* или *Post*. Класс сервлета наследуется от *HttpServlet* и содержит методы *doGet* и *doPost*; в работе оба перенаправлены на один метод *processRequest*(*HttpServletRequest request*, *HttpServletResponse response*). Значение элемента формы читает метод *request.getParameter*(«имя»), ответ клиенту записывается в поток *response.getWriter*() методом *println*, тип содержимого задает *response.setContentType*(...).

Конфигурационный файл *web.xml* регистрирует сервлеты (*servlet*, *servlet-name*, *servlet-class*), связывает их с адресами (*servlet-mapping*, *url-pattern*) и задает стартовую страницу приложения (*welcome-file*). Приложение упаковывается в архив *WAR* и размещается на сервере приложений (в работе – *Apache Tomcat* 9). Клиентом может быть и обычное *Java*-приложение, которое обращается к сервлету по *URL*, например с помощью библиотеки *Apache HttpClient*.

2.3 Подготовка среды и структура проекта

Серверный проект *lab_servlet* собран системой *Maven* 3.10.0 с *JDK* 17 (*Zulu* 17.0.15) в архив *WAR* командой *mvn package*. Архив размещен в каталоге *webapps* сервера *Apache Tomcat* 9.0.122, приложение доступно по адресу *http://localhost*:8080/*lab_servlet*/. Клиент вынесен во второй проект *apacheclient*, как требует задание. Состав проектов приведен в таблице 3.1.

Данные по теме «состав проектов» приведены в таблице 2.1.

Таблица 2.1 – Состав проектов

Проект	Файл	Назначение
Lab_servlet	*Src/main/webapp/dictionary.html*	Страница со словарем (клиентская часть)
Lab_servlet	*Src/main/java/MyServlet.java*	Сервлет перевода и обратного перевода
Lab_servlet	*Src/main/java/ImageServlet.java*	Сервлет возврата рисунка (вариант 2)
Lab_servlet	*Src/main/webapp/WEB-INF/web.xml*	Конфигурация сервлетов
Lab_servlet	*Src/main/webapp/images/picture.png*	Файл рисунка
Apacheclient	*Src/main/java/apacheclient/ApacheClient.java*	*Java*-клиент на *Apache HttpClient*

Серверный проект упаковывается в архив *WAR* и работает под управлением *Tomcat*, клиентский проект запускается отдельно как обычное приложение *Java*.

2.4 Страница *dictionary.html*

Страница создана по образцу из методических указаний: форма с методом *Get* и действием *MyServlet*, поле ввода русского слова *txt* и кнопка отправки. Для обратного перевода добавлена вторая форма с полем *eng*, для индивидуального задания – третья форма с вызовом *ImageServlet*. Код страницы приведен в приложении А.

Страница в браузере показана на рисунке 2.1.

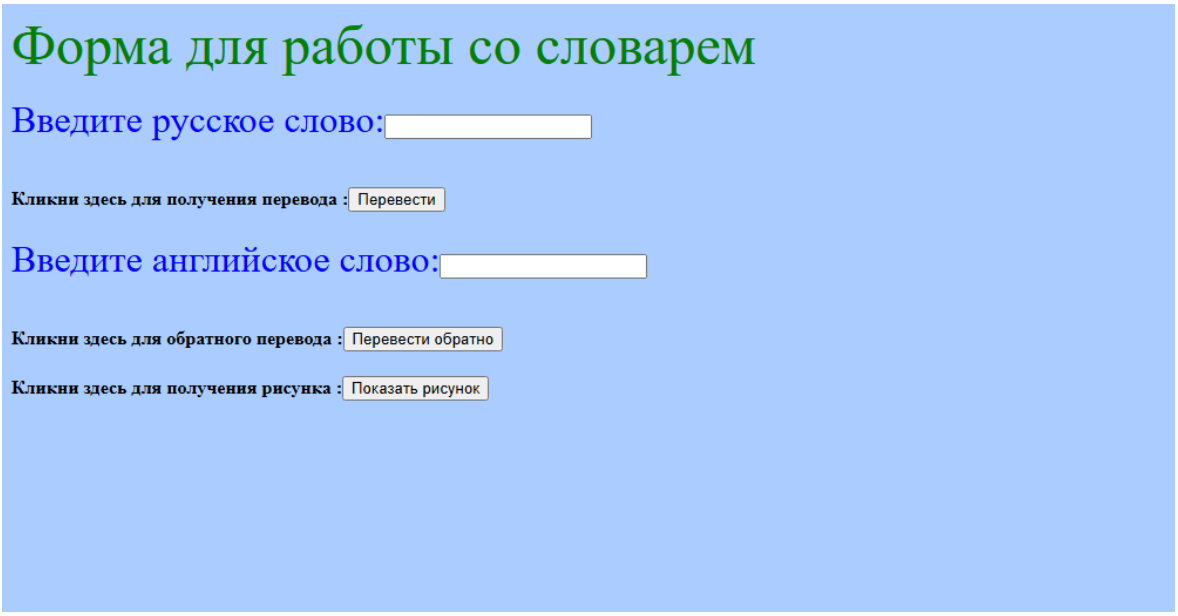


Рисунок 2.1 – Страница *dictionary.html*

Страница содержит три формы: перевод с русского языка, обратный перевод с английского и получение рисунка.

2.5 Сервлет *MyServlet* и база данных переводов

Переводы хранятся в базе данных *H2*, работающей в памяти приложения: при инициализации сервлета (метод *init*()) создается таблица *words* со столбцами *rus* и *eng* и добавляются 10 пар слов. Метод *processRequest* читает параметр *txt* (русское слово) или *eng* (английское слово), выполняет запрос *SELECT* к таблице и выводит результат в *html*-страницу. Для обратного перевода используется тот же запрос с обратным порядком столбцов. Код приведен в приложении А.

Обработка запроса и формирование ответа приведены в следующем листинге.

Сервлеты регистрируются в файле *web.xml*, структура которого приведена в приложении А. В нем заданы имена и классы сервлетов, их адреса */MyServlet* и */ImageServlet* и стартовая страница *dictionary.html*.

2.6 Работа словаря

Перевод слова «привет» показан на рисунке 2.2.



Рисунок 2.2 – Перевод с русского языка на английский

Сервлет вернул страницу с результатом «привет - *hello*». Параметр передается методом *Get*, адрес запроса содержит *MyServlet*?*txt*=привет в закодированном виде.

Обратный перевод слова *book* показан на рисунке 2.3.



Рисунок 2.3 – Обратный перевод с английского языка на русский

Результат «*book* - книга» получен тем же сервлетом по параметру *eng*.

Если слова нет в базе данных, сервлет сообщает об этом, как показано на рисунке 2.4.

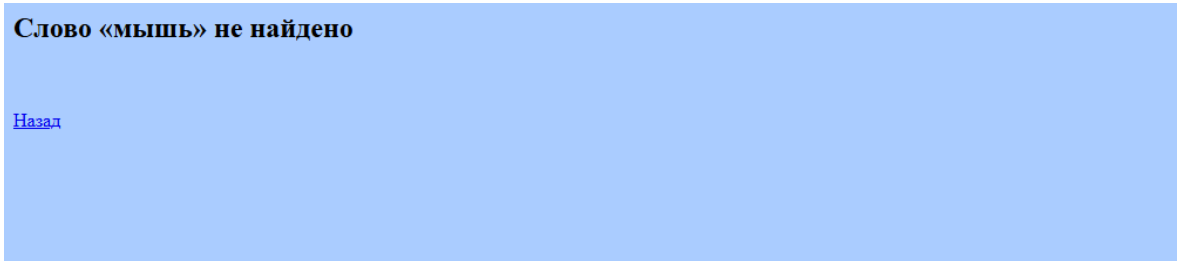


Рисунок 2.4 – Слово отсутствует в словаре

Для слова «мышь» выведено сообщение, что слово не найдено.

2.7 Клиент в отдельном проекте

Клиентское приложение *ApacheClient* из проекта *apacheclient* обращается к сервлету по адресу *http://localhost*:8080/*lab_servlet/MyServlet*

методом *HttpGet* библиотеки *Apache HttpClient* 4.5.14. Слово передается параметром *txt* или *eng* с кодированием *URLEncoder*, результат берется из тега *h2 html*-ответа сервлета. Интерфейс построен на *Swing*. Код приведен в приложении А.

Окно клиента показано на рисунке 2.5.

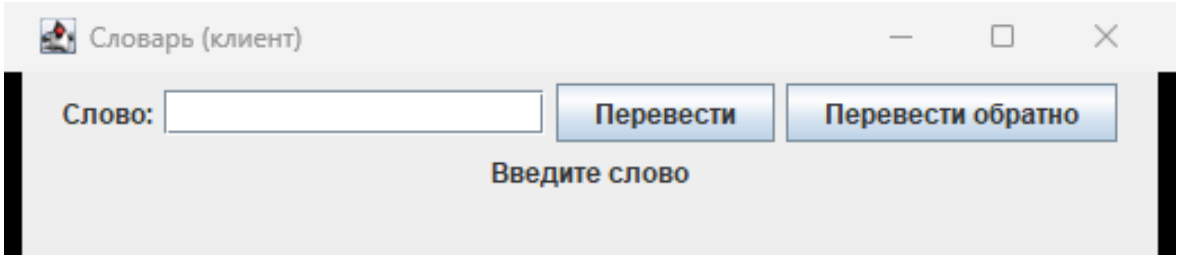


Рисунок 2.5 – Окно клиента после запуска

Окно содержит поле ввода слова, кнопки «Перевести» и «Перевести обратно» и надпись с результатом.

Перевод слова «дом» через клиент показан на рисунке 2.6.

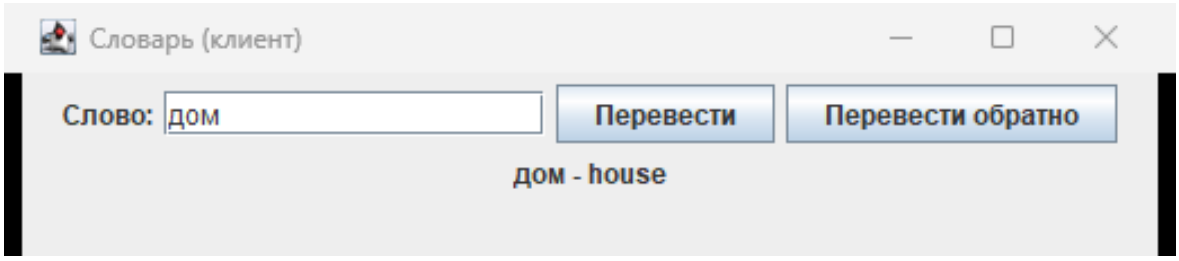


Рисунок 2.6 – Перевод слова через клиент

Клиент отправил запрос серверу в другом проекте и показал результат «дом - *house*».

Обратный перевод слова *student* показан на рисунке 2.7.

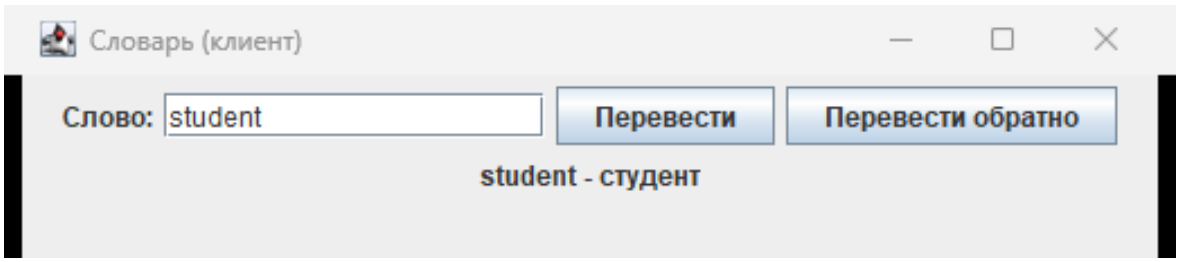


Рисунок 2.7 – Обратный перевод через клиент

Получен результат «*student* - студент».

2.8 Индивидуальное задание, вариант 2

Вариант 2: написать сервлет, возвращающий на сторону клиента файл с рисунком. Сервлет *ImageServlet* читает файл */images/picture.png* из каталога приложения методом *getResourceAsStream*, устанавливает тип содержимого *image/png* и записывает байты файла в выходной поток ответа, а браузер отображает рисунок. Код приведен в приложении А.

Результат вызова сервлета кнопкой «Показать рисунок» показан на рисунке 2.8.

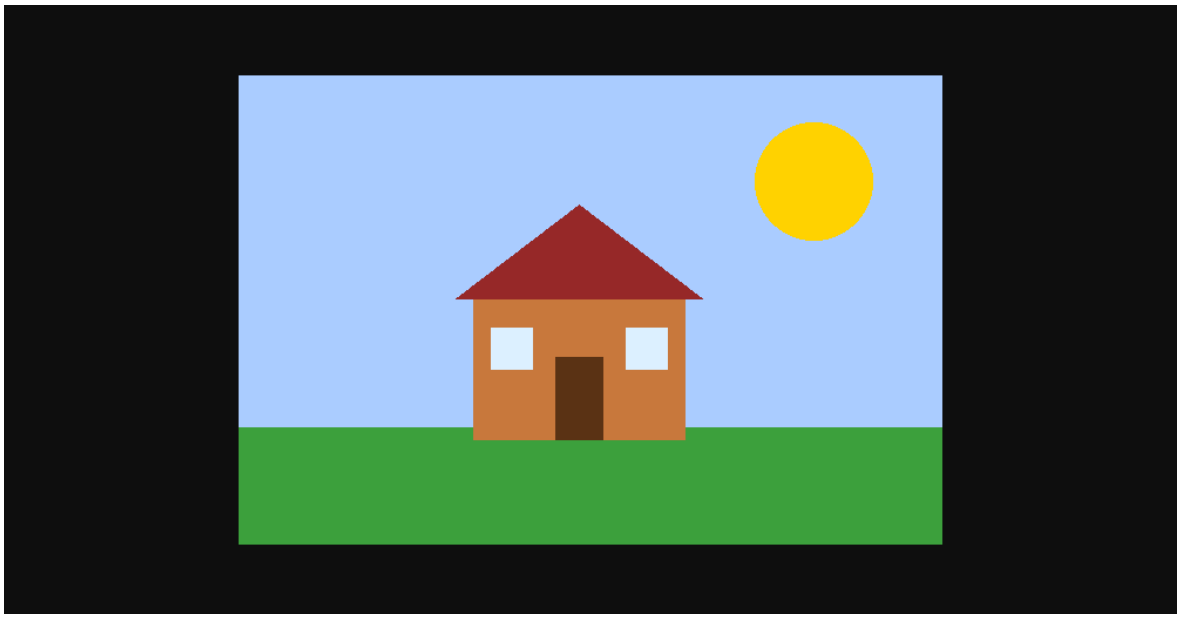


Рисунок 2.8 – Рисунок, возвращенный сервлетом *ImageServlet*

Браузер получил файл *picture.png* от сервлета по адресу */lab_servlet/ImageServlet* и отобразил рисунок: дом, солнце и земля.

3 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое сервлет, какие основные объекты и методы он предоставляет?

Сервлет – класс *Java* на стороне сервера, обрабатывающий запросы клиентов. Основные методы – *doGet* и *doPost*, выбираемые по атрибуту *method* формы. Основные объекты: *HttpServletRequest* (параметры запроса, метод *getParameter*) и *HttpServletResponse* (ответ клиенту, методы *setContentType*, *getWriter*, *getOutputStream*).

2. Как сервлет определяется в клиентском сайте?

Сервлет вызывается формой *html*-документа: в теге *form* атрибут *action* содержит имя сервлета (в работе *action*="*MyServlet*"), а кнопка с типом *submit* отправляет данные формы.

3. Для чего служит конфигурационный файл *web.xml*?

Файл *web.xml* – дескриптор развертывания приложения. В теге *servlet* указываются имя и класс сервлета, в теге *servlet-mapping* – адрес (*url-pattern*), по которому он вызывается, в *welcome-file-list* – стартовая страница, в *session-config* – время жизни сессии.

4. Как вернуть результаты из сервлета на сторону клиента?

Результаты записываются в ответ: для текста и *html* используется *PrintWriter out* = *response.getWriter*() и метод *out.println*, для файлов (рисунок, документ) – *response.getOutputStream*() с установленным типом содержимого (*response.setContentType*), как в *ImageServlet*.

ВЫВОДЫ

Изучена техника использования сервлетов: созданы страница *dictionary.html*, сервлет *MyServlet* с базой данных переводов и обратным переводом, конфигурационный файл *web.xml*. Клиент вынесен в отдельный проект на *Apache HttpClient* и вызывает сервлет по *URL*. По индивидуальному заданию (вариант 2) написан сервлет *ImageServlet*, возвращающий клиенту файл с рисунком. Все функции проверены на сервере *Apache Tomcat*. Поставленная цель достигнута.

ПРИЛОЖЕНИЕ А
(ОБЯЗАТЕЛЬНОЕ)
ЛИСТИНГИ ПРОГРАММНОГО КОДА

Листинг А.1 – Файл *dictionary.html*

```
<!DOCTYPE html>
<html>
    <head>
      <title></title>
      <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
   </head>
   <body bgcolor="#aaccff">
      <Font color="green" size="10">
       Форма для работы со словарем
      </Font>
      <br>
      <br>
      <form name="frm" method="Get" action="MyServlet">

         <Font color="blue" size="6"> Введите русское слово:</Font><Input type="Text"
name="txt" value=""/>
         <br>
         <br>
      <h4>Кликни здесь для получения перевода :<Input type="submit"
value="Перевести"/>
      </h4>
      </form>

      <form name="frm2" method="Get" action="MyServlet">
         <Font color="blue" size="6"> Введите английское слово:</Font><Input type="Text"
name="eng" value=""/>
         <br>
         <br>
      <h4>Кликни здесь для обратного перевода :<Input type="submit" value="Перевести
обратно"/>
      </h4>
      </form>

      <form name="frm3" method="Get" action="ImageServlet">
      <h4>Кликни здесь для получения рисунка :<Input type="submit" value="Показать
рисунок"/>
      </h4>
      </form>
   </body>
</html>
```

Листинг А.2 – Инициализация базы данных и поиск перевода

```
// Создание базы данных с переводами слов
@Override
public void init() throws ServletException {
```

```
    try {
        Class.forName("org.h2.Driver");
        connection =
DriverManager.getConnection("jdbc:h2:mem:dictionary;DB_CLOSE_DELAY=-1");
        Statement st = connection.createStatement();
        st.execute("CREATE TABLE words (rus VARCHAR(50), eng VARCHAR(50))");
        st.execute("INSERT INTO words VALUES ('привет', 'hello'), ('дом', 'house'), "
                + "('книга', 'book'), ('студент', 'student'), ('компьютер', 'computer'), "
                + "('вода', 'water'), ('солнце', 'sun'), ('друг', 'friend'), "
                + "('школа', 'school'), ('город', 'city')");
    } catch (Exception e) {
        throw new ServletException(e);
    }
}

// Поиск перевода в базе данных: from - столбец с исходным словом, to - столбец с
переводом
private String translate(String from, String to, String word) throws Exception {
    PreparedStatement ps = connection.prepareStatement(
            "SELECT " + to + " FROM words WHERE " + from + " = ?");
    ps.setString(1, word.trim().toLowerCase());
    ResultSet rs = ps.executeQuery();
    return rs.next() ? rs.getString(1) : null;
}
```

Листинг А.3 – Метод *processRequest*

```
protected void processRequest(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
    response.setContentType("text/html;charset=UTF-8");
    PrintWriter out = response.getWriter();

    String rus_word = request.getParameter("txt"); // русское слово
    String eng_word = request.getParameter("eng"); // английское слово (обратный перевод)

    String result;
    try {
        if (rus_word != null && !rus_word.trim().isEmpty()) {
            String found = translate("rus", "eng", rus_word);
            result = found == null ? "Слово «" + rus_word + "» не найдено" : rus_word + " - " +
found;
        } else if (eng_word != null && !eng_word.trim().isEmpty()) {
            String found = translate("eng", "rus", eng_word);
            result = found == null ? "Слово «" + eng_word + "» не найдено" : eng_word + " - " +
found;
        } else {
            result = "Слово не введено";
        }
    } catch (Exception e) {
        throw new ServletException(e);
    }

    out.println("<html>");
```

```
    out.println("<head>");
    out.println("<title>Servlet  MyServlet</title>");
    out.println("<meta  http-equiv=\"Content-Type\"  content=\"text/html;  charset=UTF-8\">");
    out.println("</head>");
    out.println("<body  bgcolor='#aaccff'>");
    out.println("<h2>" + result + "</h2><br><br>");
    out.println("<a  href=\"dictionary.html\">Назад</a>");
    out.println("</body>");
    out.println("</html>");
}
```

Листинг А.4 – Файл *web.xml*

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="3.0" xmlns="http://java.sun.com/xml/ns/javaee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://java.sun.com/xml/ns/javaee  http://java.sun.com/xml/ns/javaee/web-
app_3_0.xsd">
    <servlet>
        <servlet-name>MyServlet</servlet-name>
        <servlet-class>MyServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>MyServlet</servlet-name>
        <url-pattern>/MyServlet</url-pattern>
    </servlet-mapping>
    <servlet>
        <servlet-name>ImageServlet</servlet-name>
        <servlet-class>ImageServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>ImageServlet</servlet-name>
        <url-pattern>/ImageServlet</url-pattern>
    </servlet-mapping>
    <welcome-file-list>
        <welcome-file>
            dictionary.html
        </welcome-file>
     </welcome-file-list>
     <session-config>
        <session-timeout>
            30
        </session-timeout>
    </session-config>
</web-app>
```

Листинг А.5 – Клиент *ApacheClient*

```
public class ApacheClient {

    static String url = "http://localhost:8080/lab_servlet/MyServlet";

    // Отправка GET-запроса сервлету и выделение результата из html-ответа
```

```
    static String ask(String parameter, String word) throws Exception {
        try (CloseableHttpClient client = HttpClients.createDefault()) {
            HttpGet request = new HttpGet(url + "?" + parameter + "=" +
URLEncoder.encode(word, "UTF-8"));
            HttpResponse response = client.execute(request);
            String html = EntityUtils.toString(response.getEntity(), "UTF-8");
            int start = html.indexOf("<h2>") + 4;
            int end = html.indexOf("</h2>");
            return html.substring(start, end);
        }
    }

}
```

Листинг А.6 – Сервлет *ImageServlet*

```
public class ImageServlet extends HttpServlet {

    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        // Файл с рисунком находится в каталоге приложения
        InputStream in = getServletContext().getResourceAsStream("/images/picture.png");
        if (in == null) {
            response.sendError(HttpServletResponse.SC_NOT_FOUND, "Файл рисунка не
найден");
            return;
        }

        // Тип содержимого: браузер сам отображает рисунок
        response.setContentType("image/png");
        OutputStream out = response.getOutputStream();
        byte[] buffer = new byte[4096];
        int length;
        while ((length = in.read(buffer)) != -1) {
            out.write(buffer, 0, length);
        }
        in.close();
        out.close();
    }
```